

Final Project: *Conway's Game of Life*

报告人: 郑子昂

指导教师: 蔡铭

学号: 3250102126

报告日期: 2026.6.18

零、康威生命游戏简介

康威生命游戏(Conway's Game of Life)是元胞自动机的一种。在一个无限大的网格空间里,每个网格中有一个元胞,它会随周围8个元胞的状态改变自身状态。当中心元胞存活时,若其邻居中有2个或3个存活,则自身维持存活状态,否则死亡;当中心无元胞时,若周围邻居恰有3个存活,则中心元胞诞生,否则不变。

这种简单的规则可以创造一些奇妙的图案及演化现象。

更多请参考 https://en.wikipedia.org/wiki/Conway%27s_Game_of_Life

一、设计思路

本项目在FPGA上完整实现了Conway生命游戏,核心设计围绕三个问题展开。

- 网格如何存储? —— grid_bram 维护一个 64×48 的二维寄存器数组,每比特对应一个细胞的死活。采用双端口设计:Port A 以 25 MHz 供 VGA 实时只读,Port B 以 100 MHz 供计算与控制逻辑读写。两个端口互不阻塞,VGA 刷新与游戏演化并行运行。
- 细胞如何演化? —— game_core 采用“三行缓冲 + 滑动窗口”的流水线架构。使用一个“九宫格”按序遍历整个游戏界面的所有细胞单元,每次都需要中心细胞及其所有邻居地状态以执行康威生命游戏的演化逻辑。通过几个额外的行数组来降低遍历的时间复杂度。
- 用户如何交互? —— PS/2 键盘经 ps2_keyboard 解码后送入 ctrl_fsm,后者维护封面 / 编辑 / 运行三态切换。编辑态下方向键移动光标、空格翻转细胞,数字键一键加载 8 种预设经典图案。运行态下 tick_gen 以可调速脉冲驱动 game_core 逐代演化,蜂鸣器每代发一声滴答。

二、系统架构

系统以 gol_top 为顶层,将各子模块通过总线互连。整体分为三个数据通路:

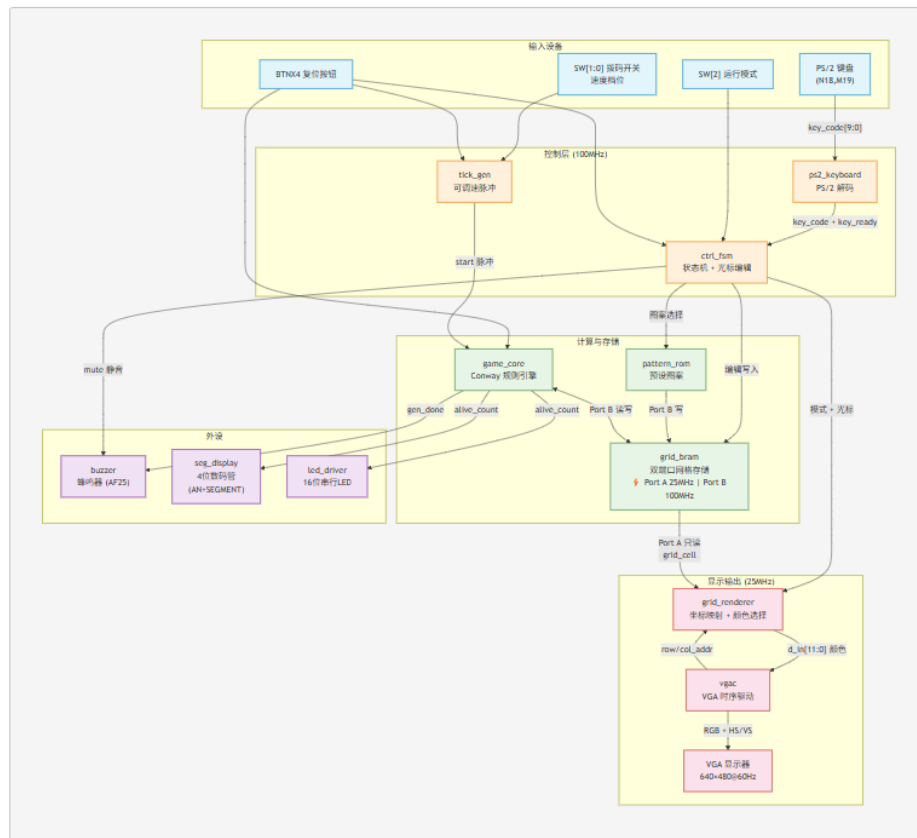


图 1 系统架构图(led 模块实际没有实现)

顶层模块 gol_top 将各子模块按如上方式串联在一起。

• 顶层模块代码与接口

```

module gol_top (
    input          clk,                // 100MHz 系统时钟 (AC18)
    input          rst_n,              // 复位按钮 (BTN04, W16)
    input  [1:0]   speed,              // SW[1:0]: 演化速度档位
    input          run_mode,           // SW[2]: 编辑/运行 模式切换

    input          ps2_clk,            // PS/2 时钟 (N18)
    input          ps2_data,           // PS/2 数据 (M19)

    output  [3:0]  vga_r, vga_g, vga_b, // VGA 颜色输出
    output          vga_hs, vga_vs,     // VGA 同步信号
    output          buzzer,             // 蜂鸣器 (AF25)
    output  [3:0]  AN,                 // 数码管位选
    output  [7:0]  SEGMENT             // 数码管段选
);

```

内部例化了 11 个子模块，各模块功能与接口详见第二节。时钟方面，仅使用板载 100 MHz 晶振，通过 32 位计数器分频产生 25 MHz VGA 像素时钟，未引入 MMCM 或 PLL IP 核。

三、模块实现

I. 基础显示

在 VGA 上显示活细胞/死细胞

4.1.1 vgac 模块

vgac 为课程提供的 VGA 时序驱动模块，负责产生 $640 \times 480 @ 60\text{Hz}$ 显示模式所需的行同步信号 (HS) 与场同步信号 (VS)，并逐像素输出当前扫描坐标，供后续模块查询对应颜色。

4.1.2 时钟方案

系统仅使用板载 100 MHz 时钟源 (引脚 AC18)，不引入 MMCM 或 PLL IP 核。VGA 显示所需的约 25 MHz 像素时钟由一个 32 位计数器分频得到：

```
reg [31:0] clkdiv;  
always @(posedge clk) begin  
    clkdiv <= clkdiv + 1'b1;  
end  
wire vga_clk = clkdiv[1];    // 100MHz / 4 = 25MHz
```

设计中存在两个时钟域：

- 系统主时钟，驱动 Game Core (Port B) 及控制逻辑 [100MHz]
- VGA 域，驱动 vgac、grid_bram (Port A)、grid_renderer [25MHz]

4.1.3 grid_bram 模块

维护一个二维数组，存储所有细胞块的生/死状态。它从后面的 game_core 模块获取细胞状态，并将状态传给 grid_renderer 翻译为颜色值进而传给 VGA。

使用两个端口 PortA 和 PortB，分别负责 VGA 的读取以及 game_core 的读取+写入。两者并行运行，使显示与计算互不冲突，且 game_core 的

时钟频率比 VGA 读取的频率要快(100MHz&25MHz)，保证在显示之前已经更新完毕。

• 代码：

```
module grid_bram (
    // Port A – VGA 只读
    input          clk_a,          // VGA 时钟域 (25MHz)
    input [5:0]    addr_a_row,     // 行地址 0~47
    input [5:0]    addr_a_col,     // 列地址 0~63
    output reg     dout_a,         // 读出数据：该格细胞状态

    // Port B – Game Core 读写
    input          clk_b,          // 系统时钟域 (100MHz)
    input [5:0]    addr_b_row,     // 行地址 0~47
    input [5:0]    addr_b_col,     // 列地址 0~63
    input          we_b,           // 写使能
    input          din_b,          // 写入数据
    output reg     dout_b         // 读出数据
);

// 网格存储：48 行 × 64 列 × 1 bit
reg [0:0] grid [0:47][0:63];

// 初始化
integer init_r, init_c;
initial begin
    for (init_r = 0; init_r < 48; init_r = init_r + 1)
begin
        for (init_c = 0; init_c < 64; init_c = init_c +
1) begin
            grid[init_r][init_c] = 1'b0;
        end
    end
end

// Port A: VGA 读取 (同步读)
```

```

always @(posedge clk_a) begin
    dout_a <= grid[addr_a_row][addr_a_col];
end

// Port B: Game Core 读写 (同步读写)
always @(posedge clk_b) begin
    if (we_b) begin
        grid[addr_b_row][addr_b_col] <= din_b;
    end
    dout_b <= grid[addr_b_row][addr_b_col];
end

endmodule

```

• 仿真

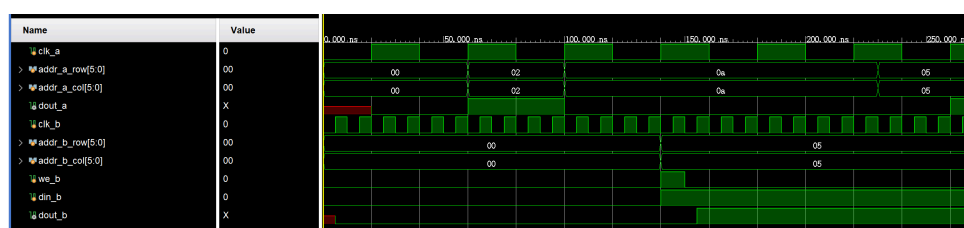


图 2 grid_bram 仿真波形图

从图中可以看出，将(2,2)细胞颜色值赋为 1，dout_a 也跟着变为 1 (之后会被 VGA 读取)。而当 din_b 变为 1，且使能也为 1 时，模块在 clk_b 的上升沿读到该值，并在下一个时钟上升沿将 PortB 正在扫描的(5,5)细胞的颜色值更新为 1 (可从 dout_b 看出，隔了一个时钟周期是因为非阻塞赋值保证了同一时钟周期，读永远发生在写之前，所以 dout_b 不能立即更新)，而当 PortA 之后扫描到(5,5)时，观察到 dout_a 为 1，表明 VGA 读取到游戏逻辑更新的颜色值

4.1.4 grid_renderer 模块

纯组合逻辑，作为连接 VGA 和 grid_bram 的桥梁(在顶层模块中体现)，接受 VGA 当前扫描的像素坐标，将像素坐标转换成其所属的细胞块的坐标，传给 grid_bram，得到该细胞的颜色值(0/1)，并转换为实际的颜色值。

• 代码

```

module grid_renderer (
    input  [8:0]    row_addr,          // 当前扫描行 0~479
    input  [9:0]    col_addr,          // 当前扫描列 0~639
    input          rdn,                // 0=有效显示区, 1=消隐区

    // Grid BRAM 接口
    output [5:0]    grid_row,          // 要查询的细胞行 0~47
    output [5:0]    grid_col,          // 要查询的细胞列 0~63
    input          grid_cell,          // BRAM 返回的细胞状态

    input          title_on,           // 封面模式
    input  [5:0]    cursor_row,        // 待编辑细胞单元地址
    input  [5:0]    cursor_col,

    // 颜色输出给 vgac
    output [11:0]   d_out
);

// 1. 像素坐标 → 细胞网格坐标 (每个细胞占 10×10 像素)
assign grid_row = row_addr / 6'd10;    // 0~479 → 0~47
assign grid_col = col_addr / 6'd10;    // 0~639 → 0~63

// 2. 输出颜色 (优先级: 消隐 > 网格线 > 活细胞 > 死细胞)
// 网格线画在 10×10 格子边界上
wire on_grid_line = !title_on && ((row_addr % 5'd10 == 0) || (col_addr % 5'd10 == 0));
wire is_cursor    = (grid_row == cursor_row) &&
(grid_col == cursor_col); // 判断细胞单元是否可编辑

assign d_out = rdn          ? 12'h000 : // 消隐 → 黑
                is_cursor   ? 12'hF88 : // 光标 → 粉红
                title_on    ? (grid_cell ? 12'hFFF :
12'h000) : // 封面: 白字黑底
                on_grid_line ? 12'h666 : // 网格线 → 灰
                grid_cell    ? 12'h0F0 : // 活细胞 → 绿
                               12'h111; // 死细胞 → 灰黑

```

```
endmodule
```

• 仿真

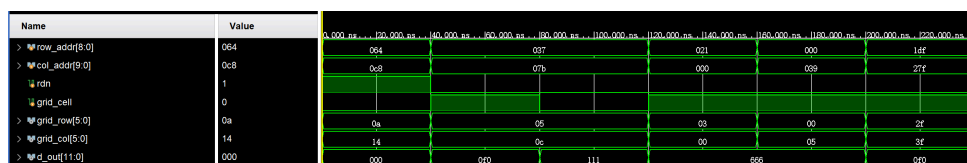


图 3 grid_renderer 仿真波形图

从图中可以看出，d_out 显示颜色值的逻辑与模块中的条件语句相同，表明模块工作正常

II. 游戏逻辑

4.2.1 game_core 模块

实现康威生命游戏的基本逻辑：当细胞为死亡状态时，如果它的邻居中（九宫格）恰好有三个存活，则该细胞下一个状态由死亡变为存活；当细胞为存活状态时，如果它的邻居中恰有两个或三个存活的细胞，则它保持不变，否则该细胞死亡。

基本思想是：遍历图中每一个细胞单元，根据其邻居及自身的状态计算它的下一个状态，并返回给 grid_bram。

- 遍历：由于遍历每个细胞的时候都需要知道它八个邻居的生死状态，如果遍历每个细胞的时候都要同时遍历它的邻居，那么效率会很低。这里，我通过把遍历单元从点封装为九宫格来解决。我主要维护了三个数组 buf_top, buf_mid, buf_bot 来存储三行细胞的全部状态信息，每一轮按列遍历中间行的所有细胞，获得其下一个状态（由于下一轮遍历需要的是细胞的上一个状态，所以不能直接在遍历数组里更新状态，而要用新的数组存储），轮换时九宫格向下移动一行，更新 buf_top, buf_mid 和 buf_bot。这里的逻辑比较复杂，在实际代码里，将遍历过程分为七个阶段，如 PREREAD, READ_NEXT, COMPUTE 等，这样更加清晰，您可以参照代码注释理解。
- 计算：定义了一个函数 cell_next 来按照如上所述的生命游戏逻辑得到细胞的下一个状态，具体使用条件语句实现。

代码：

```

module game_core (
    input          clk,
    input          rst_n,    //复位按钮，顶层模块给出
    input          start,    //start脉冲，用于控制game_core的迭
代速度，由tick_gen模块给出

    output [5:0]    bram_row,    //给BRAM更新的细胞地址
    output [5:0]    bram_col,
    output          bram_we,     //写使能
    output          bram_din,    //给BRAM写的新值
    input          bram_dout,    //BRAM传过来的旧值

    output reg      gen_done     //迭代是否结束
    output [11:0]   alive_count  // 活细胞数量
);

//遍历过程中的各个阶段
localparam [2:0]
    IDLE      = 3'd0,    // 初始状态
    PREREAD0  = 3'd1,    // 预读 row 0 → buf_mid
    PREREAD1  = 3'd2,    // 预读 row 1 → buf_bot
    READ_NEXT = 3'd3,    // 读 row r+2 → buf_temp (读新的
行)
    COMPUTE   = 3'd4,    // 计算buf_mid的64 列细胞的下一个状
态
    WRITEBACK = 3'd5,    // 写上一行 + 缓冲轮换 buf_temp →
buf_bot → buf_mid → buf_top
    CLEANUP   = 3'd6;    // 清空缓存同时写最后一行

    reg [2:0] state;

// 计数用
    reg [5:0] row;        //行进度
    reg [6:0] col;        //列进度
    reg      first;       // 每轮 BRAM 读的第一个周期

```

```

// 行缓冲
reg [63:0] buf_top;
reg [63:0] buf_mid;
reg [63:0] buf_bot;
reg [63:0] buf_temp;    //缓存下一行
reg [63:0] result_delay; //缓存这一行的result
reg [63:0] result_next;  //上一行的result
reg [11:0] alive_cnt;    //计数活细胞数量
assign alive_count = alive_cnt;

// BRAM 接口
wire do_read  = (state == PREREAD0) || (state ==
PREREAD1)      //BRAM读取
                || (state == READ_NEXT);
wire do_write = (state == WRITEBACK && row >
6'd0)          //BRAM写入
                || (state == CLEANUP);

// 读取的地址
wire [5:0] rd_row = (state == PREREAD0) ? 6'd0
                    : (state == PREREAD1) ? 6'd1
                    : (row + 6'd2);

// 写入的地址
wire [5:0] wr_row = (state == CLEANUP) ? 6'd47 : (row -
6'd1);

assign bram_row = do_read ? rd_row : wr_row;
assign bram_col = col[5:0];
assign bram_we  = do_write;
assign bram_din = result_delay[col[5:0]];

// Conway 规则
function cell_next;
    input [63:0] top, mid, bot;
    input [5:0]  c; //column
    reg  [3:0]  cnt;
    begin
        cnt = 4'd0;    //算九宫格中的总存活数

```

```

        if (c > 0) begin
            cnt = cnt + {3'b0, top[c-1]};
            cnt = cnt + {3'b0, mid[c-1]};
            cnt = cnt + {3'b0, bot[c-1]};
        end

        if (c < 63) begin
            cnt = cnt + {3'b0, top[c+1]};
            cnt = cnt + {3'b0, mid[c+1]};
            cnt = cnt + {3'b0, bot[c+1]};
        end

        cnt = cnt + {3'b0, top[c]};
        cnt = cnt + {3'b0, bot[c]};

        cell_next = (mid[c])?(cnt==4'd2||cnt==4'd3)?
1'b1:1'b0:(cnt==4'd3)?1'b1:1'b0;
    end
endfunction

// 主状态机 + 时序逻辑
always @(posedge clk or negedge rst_n) begin //rst_n is
the reset button which is effective when it swifts from 1 to
0
    if (!rst_n) begin
        state          <= IDLE;
        row             <= 6'd0;
        col             <= 7'd0;
        first           <= 1'b0;
        buf_top         <= 64'd0;
        buf_mid         <= 64'd0;
        buf_bot         <= 64'd0;
        buf_temp        <= 64'd0;
        result_delay    <= 64'd0;
        result_next     <= 64'd0;
        alive_cnt       <= 12'd0;
    end
end

```

```

        gen_done      <= 1'b0;
    end else begin
        gen_done <= 1'b0;

        case (state)

            IDLE: begin          //初始化
                col <= 7'd0;
                first <= 1'b0;
                if (start) begin
                    row      <= 6'd0;
                    buf_top  <= 64'd0;
                    buf_temp <= 64'd0;
                    alive_cnt <= 12'd0;          // 每代重新
计数
                                                state <= PREREAD0;    //执行完毕
后进入下一阶段 (下同)

                end
            end

            PREREAD0: begin      //读第0行的数据到buf_mid
                if (!first) begin //使用first来初始化
                    col  <= 7'd1;
                    first <= 1'b1;
                end else if (col < 65) begin
                    buf_mid[col[5:0] - 1'b1] <=
bram_dout;

                    col <= col + 7'd1;
                end else begin
                    first <= 1'b0;
                    state <= PREREAD1;
                end
            end

            PREREAD1: begin      //读第1行数据到buf_bot
                if (!first) begin
                    col  <= 7'd1;

```

```

        first <= 1'b1;
    end else if (col < 65) begin
        buf_bot[col[5:0] - 1'b1] <=
bram_dout;

        col <= col + 7'd1;
    end else begin
        col <= 7'd0;
        first <= 1'b0;
        state <= READ_NEXT;
    end
end

READ_NEXT: begin          //读下一行
    if (row + 6'd2 >= 6'd48) begin
        buf_temp <= 64'd0;
        col <= 7'd0;
        first <= 1'b0;
        state <= COMPUTE;
    end else if (!first) begin
        col <= 7'd1;
        first <= 1'b1;
    end else if (col < 65) begin
        buf_temp[col[5:0] - 1'b1] <=
bram_dout;

        col <= col + 7'd1;
    end else begin
        col <= 7'd0;
        first <= 1'b0;
        state <= COMPUTE;
    end
end

COMPUTE: begin            //计算细胞的下一个状态
    if (col < 64) begin
        result_next[col[5:0]] <=
cell_next(                //存入result_next
                        buf_top, buf_mid, buf_bot,

```

```

col[5:0]
    );
    alive_cnt <= alive_cnt + {11'b0,
buf_mid[col[5:0]]}; //在此处插入活细胞数量的计数，只累加当前中心细胞
的状态

    col <= col + 7'd1;
end else begin
    col <= 7'd0;
    state <= WRITEBACK;
end
end

WRITEBACK: begin //将数据传给BRAM写入
    if (col < 64) begin
        // we_b=1 时 BRAM 自动写入 (row>0)
        col <= col + 7'd1;
    end else begin
        // 缓冲轮换
        col <= 7'd0;
        buf_top <= buf_mid;
        buf_mid <= buf_bot;
        buf_bot <= buf_temp;
        result_delay <= result_next;
        row <= row + 6'd1;

        if (row < 6'd48) begin
            state <= READ_NEXT; // 还有行
要处理

        end else begin
            state <= CLEANUP; // 全部算
完

        end
    end
end

CLEANUP: begin
    if (col < 64) begin

```

```

        col <= col + 7'd1;
    end else begin
        gen_done <= 1'b1; //结束遍历
        state <= IDLE; //恢复初始状态
    end
end
end

default: state <= IDLE;
endcase
end
end

endmodule

```

仿真：

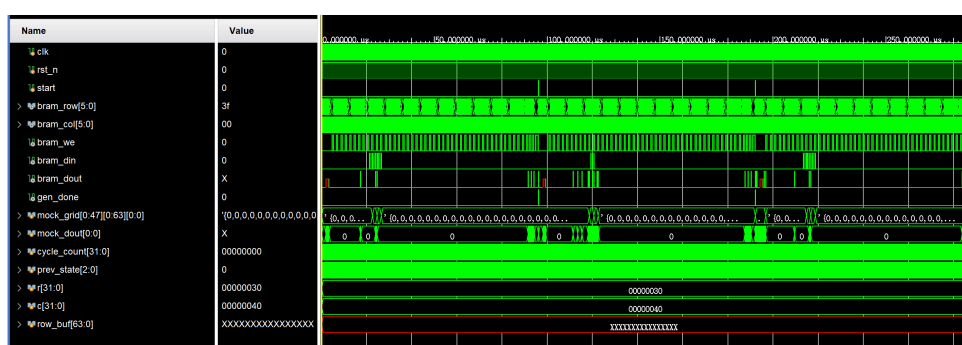


图 4 game_core 仿真波形图

<pre> ===== 验证第 1 代 ===== 列: 28 29 30 31 32 33 34 行 8: 行 9: 行 10: 行 11: 行 12: ✓ 通过: (9,31) 应诞生 ✓ 通过: (10,31) 应存活 ✓ 通过: (11,31) 应诞生 ✓ 通过: (10,30) 应死亡 (原Blinker左端, 邻居=1) ✓ 通过: (10,32) 应死亡 (原Blinker右端, 邻居=1) </pre>	<pre> ===== 验证第 2 代 (应恢复横向) ===== 列: 28 29 30 31 32 33 34 行 8: 行 9: 行 10: 行 11: 行 12: ✓ 通过: (10,30) 应恢复 ✓ 通过: (10,31) 应恢复 ✓ 通过: (10,32) 应恢复 ✓ 通过: (9,31) 应回死 ✓ 通过: (11,31) 应回死 </pre>
--	--

图 5 Tcl Console 的调试输出 (选取了若干测试点)

4.2.2 tick_gen 模块

对输入时钟信号分频(系统时钟域 100MHz), 具体通过设定计数器的阈值来实现, 最后将得到的时钟信号作为 game_core 的 start 脉冲。

逻辑简单且之前实验做过, 不进行仿真。

- 代码:

```

module tick_gen (
    input          clk,
    input          rst_n,
    input  [1:0]   speed,      // 速度档位0~3
    output reg     start      // 单周期脉冲
);

// 速度档位 (100MHz下)
// 0: 1 代/秒   → 计数 100_000_000
// 1: 4 代/秒   → 计数 25_000_000
// 2: 16 代/秒  → 计数 6_250_000
// 3: 64 代/秒  → 计数 1_562_500
reg [31:0] counter;
reg [31:0] threshold;

always @(*) begin
    case (speed)
        2'd0: threshold = 32'd100_000_000;
        2'd1: threshold = 32'd25_000_000;
        2'd2: threshold = 32'd6_250_000;
        2'd3: threshold = 32'd1_562_500;
    endcase
end

always @(posedge clk or negedge rst_n) begin //分频器逻辑
    if (!rst_n) begin
        counter <= 32'd0;
        start  <= 1'b0;
    end else begin
        start <= 1'b0;
        if (counter >= threshold - 1) begin
            counter <= 32'd0;
            start  <= 1'b1;
        end else begin
            counter <= counter + 32'd1;
        end
    end
end

```

```

        end
    end
end

endmodule

```

III.交互控制

4.3.1 ctrl_fsm 模块

控制游戏与外界的交互。实现标题页与游戏界面的转换、游戏中编辑态与运行态之间的切换以及编辑态中编辑细胞状态的功能，并把这些功能映射到键盘的相应按键。

- 代码

```

module ctrl_fsm (
    input          clk,
    input          rst_n,

    input  [9:0]    key_code,          //0~7位用于按键扫描码，第8位
    //表示通/断码 (0→按下，1→松开)，第9位表示扩展 (0→普通键，1→扩展键)
    input          key_ready,

    input          run_mode,           // SW[2] 备用
    output         running,

    output         title_on,           // TITLE 态
    output         cursor_on,
    output         mute,               // 蜂鸣器静音
    output [5:0]   cursor_row,
    output [5:0]   cursor_col,

    // 编辑 / 清空 写入 Port B
    output         edit_we,
    output reg     edit_data,
    output [5:0]   edit_row,
    output [5:0]   edit_col,

```

```

input          bram_dout,

// 预设图案
output reg [2:0] pattern_sel,
output reg      pattern_load,
input          pattern_done
);

// 键码定义
localparam [7:0]
    K_UP      = 8'h75,  K_DOWN  = 8'h72,  K_LEFT  = 8'h6B,
K_RIGHT = 8'h74,
    K_SPACE  = 8'h29,  K_ENTER = 8'h5A,  K_ESC   = 8'h76,
K_BS       = 8'h66,
    K_SHIFT  = 8'h12,  K_M      = 8'h3A;

// 按键解码
wire key_make = key_ready && !key_code[8]; // 仅通码

wire is_up    = key_make && key_code[7:0] == K_UP    &&
key_code[9];
wire is_down  = key_make && key_code[7:0] == K_DOWN  &&
key_code[9];
wire is_left  = key_make && key_code[7:0] == K_LEFT  &&
key_code[9];
wire is_right = key_make && key_code[7:0] == K_RIGHT &&
key_code[9];
wire is_space = key_make && key_code[7:0] == K_SPACE;
wire is_enter = key_make && key_code[7:0] == K_ENTER;
wire is_esc   = key_make && key_code[7:0] == K_ESC;
wire is_bs    = key_make && key_code[7:0] == K_BS;
wire is_shift = key_make && key_code[7:0] == K_SHIFT;
wire is_m     = key_make && key_code[7:0] == K_M;
wire is_k1    = key_make && key_code[7:0] == 8'h16;
wire is_k2    = key_make && key_code[7:0] == 8'h1E;
wire is_k3    = key_make && key_code[7:0] == 8'h26;
wire is_k4    = key_make && key_code[7:0] == 8'h25;

```

```

wire is_k5 = key_make && key_code[7:0] == 8'h2E;
wire is_k6 = key_make && key_code[7:0] == 8'h36;
wire is_k7 = key_make && key_code[7:0] == 8'h3D;
wire is_k8 = key_make && key_code[7:0] == 8'h3E;

// TITLE → EDIT → RUN
localparam [1:0] TITLE = 2'd0, EDIT = 2'd1, RUN = 2'd2;
reg [1:0] mode;

assign running    = (mode == RUN);
assign title_on   = (mode == TITLE);
assign cursor_on  = (mode == EDIT);

reg mute_r;
assign mute = mute_r;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) mute_r <= 1'b0;
    else if (is_m) mute_r <= ~mute_r;
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        mode <= TITLE;          // 启动显示封面
    else case (mode)
        TITLE: if (is_shift || is_enter) mode <= EDIT;
        EDIT:  if (is_shift)           mode <= TITLE;
               else if (is_enter)      mode <= RUN;
        RUN:   if (is_esc)              mode <= EDIT;
    endcase
end

// 光标移动 (仅 EDIT)
reg [5:0] cur_r, cur_c;

assign cursor_row = cur_r;
assign cursor_col = cur_c;

```

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        cur_r <= 6'd24;
        cur_c <= 6'd32;
    end else if (mode == EDIT) begin
        if (is_up    && cur_r > 0)      cur_r <= cur_r -
6'd1;
        if (is_down  && cur_r < 6'd47) cur_r <= cur_r +
6'd1;
        if (is_left  && cur_c > 0)      cur_c <= cur_c -
6'd1;
        if (is_right && cur_c < 6'd63) cur_c <= cur_c +
6'd1;
    end
end

// 翻转 / 清空 (共用 edit_data)
reg toggle_pending;
reg toggle_write;
reg      clearing;
reg [11:0] clear_idx;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        toggle_pending <= 1'b0;
        toggle_write   <= 1'b0;
        edit_data       <= 1'b0;
        clearing        <= 1'b0;
        clear_idx       <= 12'd0;
    end else if (is_bs && mode == EDIT) begin
        clearing <= 1'b1;
        clear_idx <= 12'd0;
        edit_data <= 1'b0;
        toggle_pending <= 1'b0;
        toggle_write   <= 1'b0;
    end else if (clearing) begin
        if (clear_idx < 12'd3072) begin

```

```

        clear_idx <= clear_idx + 12'd1;
        edit_data <= 1'b0;
    end else
        clearing <= 1'b0;
    end else if (is_space && mode == EDIT && !
toggle_pending) begin
        toggle_pending <= 1'b1;
    end else if (toggle_pending) begin
        toggle_pending <= 1'b0;
        toggle_write    <= 1'b1;
        edit_data        <= !bram_dout;
    end else begin
        toggle_write    <= 1'b0;
    end
end

// 清空期间的地址: clear_idx → (row, col)
wire [5:0] clear_row = clear_idx[11:6]; // /64
wire [5:0] clear_col = clear_idx[5:0];  // %64

// Port B 写输出 (翻转/清空 二选一)
assign edit_we  = clearing ? 1'b1 : toggle_write;
assign edit_row = clearing ? clear_row : cur_r;
assign edit_col = clearing ? clear_col : cur_c;
// edit_data 已在上面 always 块中赋值 (翻转时=!bram_dout),
清空时保持 0

// 预设图案 (EDIT + 数字键)
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        pattern_sel    <= 3'd0;
        pattern_load    <= 1'b0;
    end else if (mode == EDIT) begin
        pattern_load    <= 1'b0;
        if      (is_k1) begin pattern_sel <= 3'd1;
pattern_load <= 1'b1; end
        else if (is_k2) begin pattern_sel <= 3'd2;

```

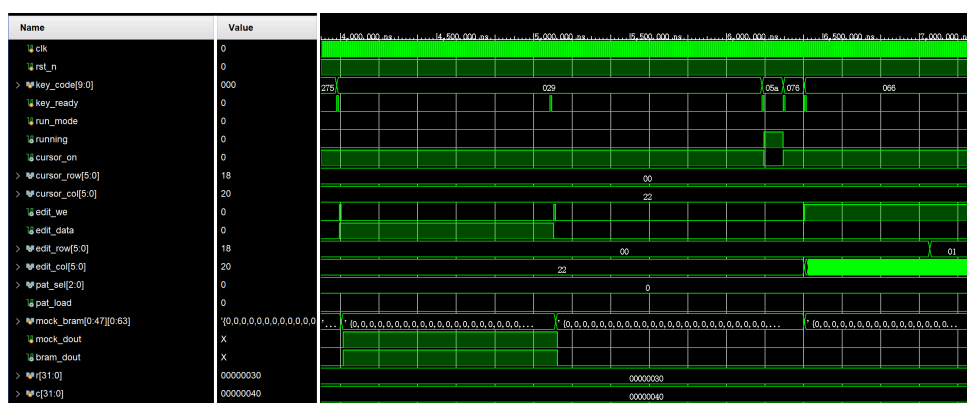



图7 ctrl_fsm 仿真波形图 2

从图中可以看出，其它按键的功能也正常。按下空格键(0x029)后，在 key_ready 上升沿时当前细胞单元的 bram_dout 反转；按下回车键(0x05a)和 ESC 键(0x076)后，running 分别变为 1 和 0，cursor_on 分别变为 0 和 1，体现了编辑态与运行态之间的切换。按下 backspace 键(0x066)后，遍历每个细胞单元，将其输出的 bram_dout 清零（过程较长只截了一部分）。

4.3.2 ps2_keyboaed 模块

键盘解码器，在每次 ps2 的时钟下降沿（即发送编码的帧）时读入键盘发送的 8bit 编码，分辨它是前缀（扩展/断码标记）还是有效码，进行解码，然后发送给 ctrl_fsm 模块。由于 ps2 时钟不被 FPGA 板视为系统时钟，而是普通输入引脚，所以需要使用移位寄存器手动检测 ps2 的下降沿。

• 代码：

```
module ps2_keyboard (
    input                clk,                // 100MHz 系统时钟
    input                rst_n,              // 复位，低有效
    input                ps2_clk,            // PS/2 时钟 (kHz 级，外部
    上拉)
    input                ps2_data,            // PS/2 数据

    output [9:0]         key_code,            // {expand(1), break(1),
    scan_code(8)}
    output               key_ready            // 单周期脉冲，key_code 有
    效
);
```

```

// PS2_CLK 下降沿检测 (2 级同步 + 边沿)
reg [1:0] clk_sync;
wire negedge_clk;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        clk_sync <= 2'd0;
    else
        clk_sync <= {clk_sync[0], ps2_clk};
end

assign negedge_clk = clk_sync[1] && !clk_sync[0]; // 打
一拍前是高, 拍后是低 = 下降沿

// 位计数器 (11 位/帧, 0→10)
reg [3:0] bit_cnt;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        bit_cnt <= 4'd0;
    else if (bit_cnt == 4'd11)
        bit_cnt <= 4'd0; // 一帧结束, 复位
    else if (negedge_clk)
        bit_cnt <= bit_cnt + 4'd1; // 每个下降沿 +1
end

// 采样数据位 (bit_cnt=2~9 为 8 位有效数据)
reg negedge_d1;
reg [7:0] scan_byte;

always @(posedge clk) begin
    negedge_d1 <= negedge_clk;
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)

```

```

        scan_byte <= 8'd0;
    else if (negedge_d1) begin                // negedge_clk
        延迟一拍, 保证 ps2_data 稳定
        case (bit_cnt)
            4'd2: scan_byte[0] <= ps2_data;
            4'd3: scan_byte[1] <= ps2_data;
            4'd4: scan_byte[2] <= ps2_data;
            4'd5: scan_byte[3] <= ps2_data;
            4'd6: scan_byte[4] <= ps2_data;
            4'd7: scan_byte[5] <= ps2_data;
            4'd8: scan_byte[6] <= ps2_data;
            4'd9: scan_byte[7] <= ps2_data;
            default: ;
        endcase
    end
end

// 输出组装 (处理 E0/F0 前缀)
reg expand;        // 扩展键标记 (收到 E0 后置 1)
reg brk;          // 断码标记 (收到 F0 后置 1)

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        expand <= 1'b0;
        brk    <= 1'b0;
    end else if (bit_cnt == 4'd11) begin
        if (scan_byte == 8'hE0) begin
            expand <= 1'b1;
            brk    <= 1'b0;
        end else if (scan_byte == 8'hF0) begin
            brk    <= 1'b1;
            expand <= 1'b0;
        end else begin
            expand <= 1'b0;
            brk    <= 1'b0;
        end
    end
end
end

```

```

end

assign key_code   = {expand, brk, scan_byte};
assign key_ready  = (bit_cnt == 4'd11) && (scan_byte !=
8'hE0) && (scan_byte != 8'hF0);

endmodule

```

• 仿真:

测试解码是否成功。选择 W 通码(随便选的普通键)、F0 前缀、W 断码以及↑通码(扩展键)

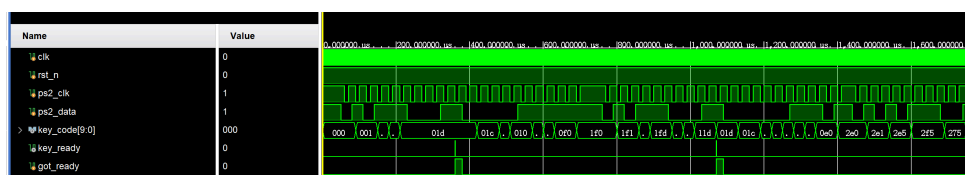


图 8 波形图

从图中可以看出, key_code 的值每一帧都发生变化(ps2_clk 下降沿), 当 key_ready 变为 1 的瞬间 key_code 为有效值, 向 ctrl_fsm 模块输出(F0 前缀不触发 key_ready, 前缀与下一帧发来的有效位共同组成键码)。通过查表可知得到的 key_code 的值与测试用例对应的编码一致。

4.3.3 pattern_rom 模块

预设置了一些经典图案，编辑时直接调用即可从当前选中细胞向周围绘制预设图案。

按键	图案	类型	细胞数
1	Glider (滑翔机)	飞船	5
2	Blinker (闪光灯)	振荡器 (周期 2)	3
3	LWSS (轻型飞船)	飞船	8
4	R-pentomino	繁衍体 (1103 代)	5
5	Acorn (橡子)	繁衍体 (5206 代)	7
6	Block (方块)	静物	4
7	Pulsar (脉冲星)	振荡器 (周期 3)	48
8	Gosper Glider Gun	滑翔机枪 (周期 30)	36

表 1 对应关系

- 代码:

```

module pattern_rom (
    input          clk,
    input          rst_n,
    input  [2:0]   pattern_sel,    // 图案选择 1~6
    input          load,           // 开始加载脉冲
    input  [5:0]   base_row,       // 光标行 (锚点)
    input  [5:0]   base_col,       // 光标列 (锚点)

    // Port B 写入接口
    output reg     write_we,
    output reg     write_data,
    output reg [5:0] write_row,
    output reg [5:0] write_col,
    output reg     done
);

// 图案定义: 每个图案由最多 40 个 (dr, dc) 坐标对组成
// dr = 行偏移, dc = 列偏移
reg [5:0] pat_dr [0:49];
reg [5:0] pat_dc [0:49];
reg [5:0] pat_len;                // 图案实际长度

reg [5:0] idx;
reg       busy;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        busy      <= 1'b0;
        idx       <= 6'd0;
        write_we  <= 1'b0;
        write_data <= 1'b0;
        write_row  <= 6'd0;
        write_col  <= 6'd0;
        done      <= 1'b0;
    end else begin
        done <= 1'b0;
    end
end

```

```

    if (load && !busy) begin
        // 选中图案，载入坐标表
        busy <= 1'b1;
        idx <= 6'd0;
        case (pattern_sel)
            3'd1: begin // Glider
                pat_dr[0]=0; pat_dc[0]=1;
pat_dr[1]=1; pat_dc[1]=2;
                pat_dr[2]=2; pat_dc[2]=0;
pat_dr[3]=2; pat_dc[3]=1; pat_dr[4]=2; pat_dc[4]=2;
                pat_len = 6'd5;
            end
            3'd2: begin // Blinker
                pat_dr[0]=0; pat_dc[0]=0;
pat_dr[1]=0; pat_dc[1]=1; pat_dr[2]=0; pat_dc[2]=2;
                pat_len = 6'd3;
            end
            3'd3: begin // LWSS
                pat_dr[0]=0; pat_dc[0]=1;
                pat_dr[1]=1; pat_dc[1]=0;
pat_dr[2]=1; pat_dc[2]=3;
                pat_dr[3]=2; pat_dc[3]=0;
                pat_dr[4]=3; pat_dc[4]=0;
pat_dr[5]=3; pat_dc[5]=1; pat_dr[6]=3; pat_dc[6]=2;
pat_dr[7]=3; pat_dc[7]=3;
                pat_len = 6'd8;
            end
            3'd4: begin // R-pentomino
                pat_dr[0]=0; pat_dc[0]=1;
pat_dr[1]=0; pat_dc[1]=2;
                pat_dr[2]=1; pat_dc[2]=0;
pat_dr[3]=1; pat_dc[3]=1;
                pat_dr[4]=2; pat_dc[4]=1;
                pat_len = 6'd5;
            end
            3'd5: begin // Acorn

```

```

        pat_dr[0]=0; pat_dc[0]=1;
        pat_dr[1]=1; pat_dc[1]=3;
        pat_dr[2]=2; pat_dc[2]=0;
pat_dr[3]=2; pat_dc[3]=1; pat_dr[4]=2; pat_dc[4]=4;
pat_dr[5]=2; pat_dc[5]=5; pat_dr[6]=2; pat_dc[6]=6;
        pat_len = 6'd7;
    end
    3'd6: begin // Block
        pat_dr[0]=0; pat_dc[0]=0;
pat_dr[1]=0; pat_dc[1]=1;
        pat_dr[2]=1; pat_dc[2]=0;
pat_dr[3]=1; pat_dc[3]=1;
        pat_len = 6'd4;
    end
    3'd7: begin // Pulsar (13x13, period 3,
48 cells)
        // Row 0: cols 2,3,4,8,9,10
pat_dr[0]=0;pat_dc[0]=2;pat_dr[1]=0;pat_dc[1]=3;pat_dr[2]=0;pat_dc[2]=4
        // Row 2: cols 1,6,8,12
pat_dr[6]=2;pat_dc[6]=1;pat_dr[7]=2;pat_dc[7]=6;pat_dr[8]=2;pat_dc[8]=8
        // Row 3: cols 1,6,8,12
pat_dr[10]=3;pat_dc[10]=1;pat_dr[11]=3;pat_dc[11]=6;pat_dr[12]=3;pat_dc
        // Row 4: cols 1,6,8,12
pat_dr[14]=4;pat_dc[14]=1;pat_dr[15]=4;pat_dc[15]=6;pat_dr[16]=4;pat_dc
        // Row 5: cols 2,3,4,8,9,10
pat_dr[18]=5;pat_dc[18]=2;pat_dr[19]=5;pat_dc[19]=3;pat_dr[20]=5;pat_dc
        // Row 7: cols 2,3,4,8,9,10
pat_dr[24]=7;pat_dc[24]=2;pat_dr[25]=7;pat_dc[25]=3;pat_dr[26]=7;pat_dc
        // Row 8: cols 1,6,8,12
pat_dr[30]=8;pat_dc[30]=1;pat_dr[31]=8;pat_dc[31]=6;pat_dr[32]=8;pat_dc
        // Row 9: cols 1,6,8,12
pat_dr[34]=9;pat_dc[34]=1;pat_dr[35]=9;pat_dc[35]=6;pat_dr[36]=9;pat_dc
        // Row 10: cols 1,6,8,12

```

```

pat_dr[38]=10;pat_dc[38]=1;pat_dr[39]=10;pat_dc[39]=6;pat_dr[40]=10;pat
    // Row 12: cols 2,3,4,8,9,10

pat_dr[42]=12;pat_dc[42]=2;pat_dr[43]=12;pat_dc[43]=3;pat_dr[44]=12;pat
    pat_len = 6'd48;
end
3'd8: begin // Gosper Glider Gun (36
cells, period 30)
    pat_dr[0]=0;pat_dc[0]=24;

pat_dr[1]=1;pat_dc[1]=22;pat_dr[2]=1;pat_dc[2]=24;
pat_dr[3]=2;pat_dc[3]=12;pat_dr[4]=2;pat_dc[4]=13;pat_dr[5]=2;pat_dc[5]
pat_dr[9]=3;pat_dc[9]=11;pat_dr[10]=3;pat_dc[10]=15;pat_dr[11]=3;pat_dc
pat_dr[15]=4;pat_dc[15]=0;pat_dr[16]=4;pat_dc[16]=1;pat_dr[17]=4;pat_dc
pat_dr[21]=5;pat_dc[21]=0;pat_dr[22]=5;pat_dc[22]=1;pat_dr[23]=5;pat_dc
pat_dr[29]=6;pat_dc[29]=10;pat_dr[30]=6;pat_dc[30]=16;pat_dr[31]=6;pat_c
pat_dr[32]=7;pat_dc[32]=11;pat_dr[33]=7;pat_dc[33]=15;
pat_dr[34]=8;pat_dc[34]=12;pat_dr[35]=8;pat_dc[35]=13;
    pat_len = 6'd36;
end
default: begin
    pat_len = 6'd0;
end
endcase
end else if (busy) begin
    if (idx < pat_len) begin
        // 在锚点位置写入活细胞
        write_we    <= 1'b1;
        write_data  <= 1'b1;
        write_row   <= base_row + pat_dr[idx];
        write_col   <= base_col + pat_dc[idx];
        idx         <= idx + 6'd1;
    end else begin
        write_we <= 1'b0;
        busy     <= 1'b0;
    end
end

```

```

done      <= 1'b1;
idx       <= 6'd0;

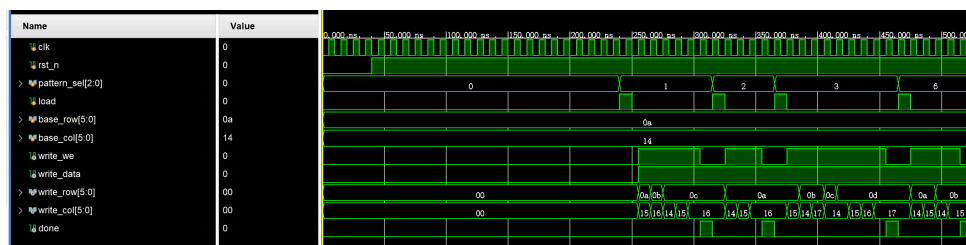
end
end
end
end

endmodule

```

• 仿真：

检测了 1, 2, 3, 6 四个图案



```

begin
    case (d) // 将十
        六进制数字映射到七段数码管
            4'h0: seg = 8'h3F; 4'h1: seg = 8'h06;
            4'h2: seg = 8'h5B; 4'h3: seg = 8'h4F;
            4'h4: seg = 8'h66; 4'h5: seg = 8'h6D;
            4'h6: seg = 8'h7D; 4'h7: seg = 8'h07;
            4'h8: seg = 8'h7F; 4'h9: seg = 8'h6F;
            4'hA: seg = 8'h77; 4'hB: seg = 8'h7C;
            4'hC: seg = 8'h39; 4'hD: seg = 8'h5E;
            4'hE: seg = 8'h79; 4'hF: seg = 8'h71;
        endcase
    end
endfunction

reg [16:0] cnt;
wire tick = (cnt == 17'd100_000); //cnt计满100000产
生tick

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) cnt <= 17'd0;
    else if (tick) cnt <= 17'd0;
    else cnt <= cnt + 17'd1;
end

reg [1:0] digit;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) digit <= 2'd0;
    else if (tick) digit <= digit + 2'd1; //根据tick改
变digit的值
end

wire [3:0] val = (digit == 0) ?
alive_count[3:0] : //分别显示三位数字
    (digit == 1) ? alive_count[7:4] :
    (digit == 2) ? alive_count[11:8] :
4'h0;

```

```

always @(posedge clk) segment <= seg(val);

always @(*) begin    //根据digit的变化，动态扫描数码管，轮流点
    亮，低电平有效
    an = 4'b1111;
    an[digit] = 1'b0;
end
endmodule

```

- 仿真：

显示活细胞数 0x003 和 0x030

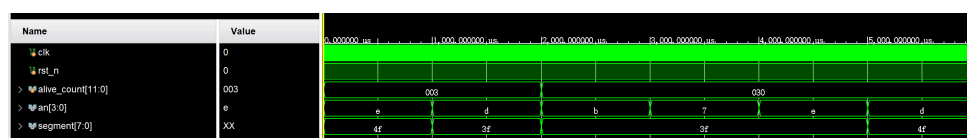


图 10 seg_display 仿真波形图

从图中可以看到输入活细胞数为 0x003 时，an 为 e(4'b1110,扫描最低位数码管)segment 为 4f(显示数字 3)，an 为 d(4'b1101)时显示数字 0(3f)，符合预期。输入活细胞数为 0x030 时，同样读 an 扫描的数码管以及对应 segment 的值，可得四个数码管显示 0030，符合预期。

4.3.5 buzzer 模块

蜂鸣器驱动模块，细胞演化每迭代一次可发出滴答声。在 ctrl_fsm 模块中设置了可通过键盘 M 键开启/关闭蜂鸣器功能。

- 代码：

```

module buzzer (
    input          clk,          // 100MHz
    input          rst_n,
    input          gen_done,     // 一代完成脉冲（来自
game_core)
    output reg     beep
);

// 1kHz 方波 = 周期 1ms → 100MHz / 100k = 1000 计数
// 半个周期 = 50000 计数

```

```

    reg [15:0] tone_cnt;
    reg [23:0] dur_cnt;          // 持续时间: 100ms → 10M
计数
    reg        playing;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            playing <= 1'b0;
            dur_cnt <= 24'd0;      // 计数器, 决定声音持续
时长
            tone_cnt <= 16'd0;     // 计数器, 决定声音的音
调
            beep <= 1'b0;         // 表示蜂鸣器是否发声
        end else begin
            if (gen_done) begin    // game_core每迭代一周
期传来的信号
                playing <= 1'b1;   // 蜂鸣器开始工作
                dur_cnt <= 24'd0;
                tone_cnt <= 16'd0;
            end else if (playing) begin
                if (dur_cnt < 24'd10_000_000) begin //
100ms
                    dur_cnt <= dur_cnt + 24'd1;
                    if (tone_cnt < 16'd50000) begin
                        tone_cnt <= tone_cnt + 16'd1;
                    end else begin
                        tone_cnt <= 16'd0;
                        beep <= ~beep;      // tone_cnt决
定beep的频率
                    end
                end else begin
                    playing <= 1'b0;
                    beep <= 1'b0;
                end
            end
        end
    end
end

```

```
endmodule
```

- 仿真:

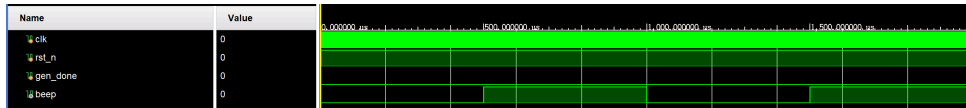


图 11 buzzer 仿真波形图

从图中可以看到 beep 的变化(由于比例问题 gen_done 只在开始的时候瞬间变 1, 可能看的不是很清晰)

四、上板操作(详见演示视频)

下面展示一些经典图案

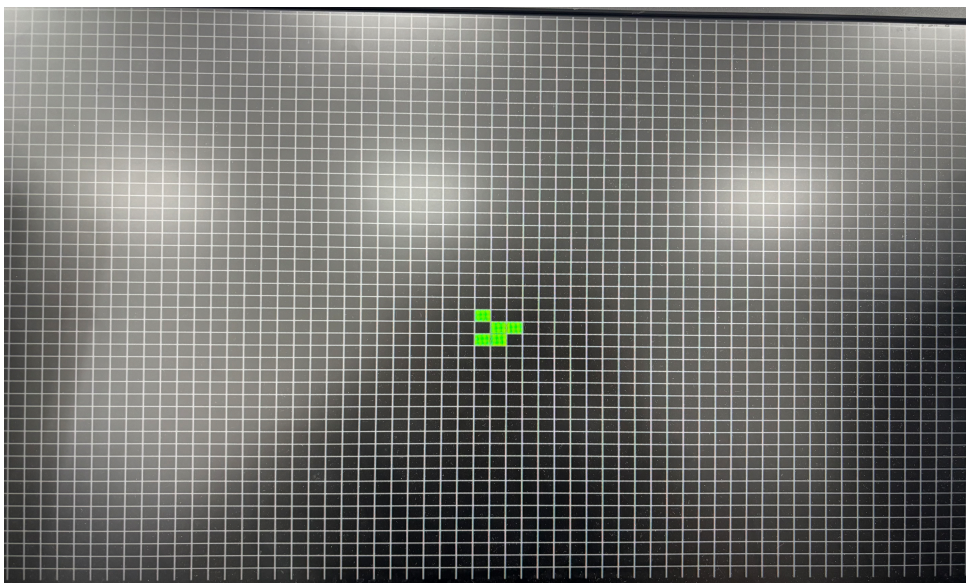


图 12 滑翔机

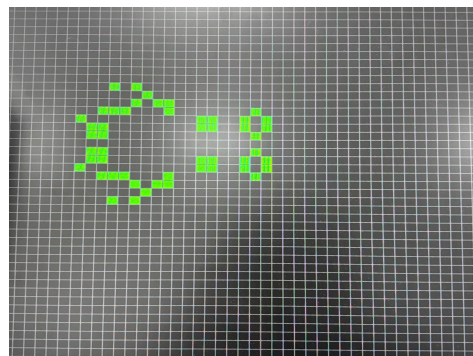


图 13 脉冲星



图 14 滑翔机枪

五、参考资料/工程

- https://en.wikipedia.org/wiki/Conway%27s_Game_of_Life (维基百科)
- <https://pages.hmc.edu/harris/class/e158/10/proj2/cardpropato.pdf>
- <https://github.com/ashkan-khd/conways-game-of-life-verilog>